Guia de estudo da API e do SQLite

Biblioteca Virtual | Código analisado em 05/11/2026

Material para o fim de semana de 07 e 08/11/2026. As explicações se referem à implementação analisada nesta data, após a inclusão do CRUD de livros e exemplares. As questões são abertas para você responder consultando o projeto e depois tentar explicar sem olhar.

1. Plano para estudar no fim de semana

Reserve 6 horas no sábado e 6 horas no domingo, incluindo leitura, execução e respostas. Distribua as pausas conforme sua necessidade. Como você escolheu dedicar o fim de semana ao estudo, a conexão do front prevista no cronograma precisará ser retomada depois desse bloco.

Sábado, 07/11 - inicialização, dados e regras

- 1h - Leia `server.ts`, `app.ts`, `config/ambiente.ts` e `config/banco.ts`. Desenhe a ordem de inicialização.
- 1h30 - Leia `database/schema.ts`, `seed.ts` e `dados_inicias.ts`. Relacione Livro, Exemplar e as tabelas.
- 1h30 - Compare `RepositorioAcervo`, `RepositorioMemoria` e `RepositorioSQLite`. Acompanhe uma consulta e uma gravação.
- 1h - Leia `servico-acervo.ts` e `validacao-acervo.ts`. Siga a criação e a atualização de um livro.
- 1h - Responda às questões dos capítulos 2 a 5 e explique em voz alta o caminho de uma consulta.

Domingo, 08/11 - HTTP, testes e explicação

- 1h - Leia rotas, controller, erros e middlewares.
- 1h30 - Execute os três rastreamentos do capítulo 7, anotando a entrada e a saída de cada camada.
- 1h30 - Leia os testes novos e faça os exercícios práticos do capítulo 9.
- 1h - Responda às questões de revisão sem consultar o código. Confira depois.
- 1h - Faça uma apresentação de 5 minutos e use o restante para revisar dúvidas.

Como trabalhar em cada arquivo

Leia os imports, identifique o que o arquivo recebe e devolve e encontre quem o chama. Execute uma operação relacionada. Anote uma frase sobre a responsabilidade do arquivo e uma dúvida concreta. Se fizer uma alteração experimental, confira o diff e reverta somente sua experiência ao terminar.

Critério de conclusão do estudo. Conseguir rastrear uma consulta, um cadastro inválido e uma exclusão bloqueada, apontando o arquivo responsável por cada decisão. Ter os arquivos copiados e os testes aprovados é uma etapa separada dessa compreensão.

2. Inicialização e arquitetura da aplicação

`server/src/server.ts` é o ponto de entrada. Ele abre o banco, chama o seed, cria `RepositorioSQLite`, passa essa instância para `criarApp` e executa `app.listen`. O tratamento do evento de erro explica, por exemplo, quando a porta já está ocupada.

`config/ambiente.ts` lê `PORT` e `DB_PATH` das variáveis de ambiente. Sem configuração externa, usa porta 3000 e `database/biblioteca.db`. O caminho do arquivo é relativo ao diretório de execução, por isso execute os scripts dentro de `server/`.

`app.ts` monta o Express. Ele configura CORS, leitura do JSON, apresentação em `/api`, saúde em `/api/saude`, rotas de livros, resposta para rota inexistente e middleware de erros. A ordem importa porque a requisição percorre os middlewares na sequência registrada.

`express.json()` interpreta o corpo JSON e o coloca em `req.body`. Um JSON malformado pode falhar aqui, antes de chegar ao controller. CORS acrescenta cabeçalhos para acesso pelo navegador entre origens diferentes. A autorização de administrador será uma funcionalidade própria, ainda pendente.

Receber uma dependência pelo construtor

`criarApp(repositorio)` recebe o armazenamento que deverá usar. Dentro dela, `ServicoAcervo` recebe esse mesmo repositório, e `LivrosController` recebe o serviço. Esse fornecimento de objetos é injeção de dependência.

O serviço depende da interface `RepositorioAcervo`. Nos testes, é possível fornecer memória ou SQLite. O código de regras permanece utilizável com as duas implementações. Essa possibilidade de usar implementações diferentes sob um mesmo contrato exemplifica polimorfismo.

O fluxo principal de uma requisição é o seguinte.

```
requisição -> rota -> controller -> serviço -> repositório -> SQLite
```

A resposta percorre o caminho de volta. Se uma operação lançar um erro, o tratamento HTTP centralizado escolhe a resposta correspondente.

Questões para responder

1. Por que o projeto separou `server.ts` de `app.ts`?
2. Qual objeto concreto é passado para `criarApp` na inicialização normal?
3. O que muda nos testes quando você troca SQLite por memória?
4. Por que um erro de JSON pode acontecer antes do controller?
5. O que `/api/saude` confirma? A implementação dessa rota executa uma consulta ao banco?

3. Banco, esquema e dados iniciais

`config/banco.ts` abre uma conexão com `better-sqlite3`. Para banco em arquivo, cria a pasta necessária. Ativa `foreign_keys = ON` e executa `SCHEMA_SQL`. O caminho `:memory:` cria um banco temporário ligado àquela conexão.

`database/schema.ts` contém os comandos de criação das tabelas. `IF NOT EXISTS` permite executar esses comandos novamente sem recriar tabelas que já existem. Alterações futuras na estrutura de tabelas existentes exigirão um procedimento próprio de migração.

As cinco tabelas

- `livros` guarda título, autor, ISBN, ano, gênero, sinopse e capa. Um livro representa a obra ou edição cadastrada.
- `exemplares` guarda as unidades do livro. Cada unidade tem ID, livro associado, código de tombo e status.
- `usuarios` prepara o armazenamento de nome, e-mail, hash da senha e papel.
- `emprestimos` prepara usuário, exemplar, datas e quantidade de renovações.
- `reservas` prepara usuário, livro, data e status da reserva.

As três últimas tabelas antecipam funcionalidades. Nesta versão, as rotas implementadas são as do acervo, além de apresentação e saúde.

`PRIMARY KEY` identifica uma linha. `UNIQUE` impede duplicidade de ISBN ou tombo. `CHECK` limita valores, como os status aceitos. Uma chave estrangeira exige que o registro relacionado exista. `COLLATE NOCASE` torna certas comparações insensíveis a maiúsculas. Os índices auxiliam consultas pelas colunas relacionadas.

`ON DELETE CASCADE` remove os exemplares associados quando o livro é excluído. A regra que bloqueia excluir exemplares em uso fica no serviço e é executada antes dessa exclusão.

Seed e reset

`dados_inicias.ts` cria um acervo novo a cada chamada, com 9 livros e 19 exemplares. Inclui caminhos de capas e ISBNs fictícios. `seed.ts` insere esses objetos no banco com uma transação. Se a gravação falhar, a transação desfaz as inserções daquela operação.

`semearSeVazio` verifica se a tabela de livros está vazia. Se já houver livros, não reaplica os dados. Editar o seed, portanto, não altera os registros de um banco já populado. Os status emprestados do acervo inicial são demonstrativos e ainda não têm registros correspondentes em `emprestimos`.

`npm run db:reset` executa `resetar.ts`, que apaga o arquivo configurado e recria o banco com os dados iniciais. Pare a API antes de um reset de desenvolvimento e confirme que o banco pode ser descartado.

Questões para responder

6. Qual é a diferença entre um livro e seus exemplares? Dê um exemplo com duas unidades.
7. O que acontece ao inserir um exemplar com `livro_id` inexistente?
8. Por que é necessário ativar as chaves estrangeiras no SQLite?
9. O que uma transação protege durante a população inicial?
10. Por que alterar `dados_inicias.ts` pode não mudar a resposta de um banco já populado?

4. Domínio e repositórios

`domain/livro.ts` ganhou `capa`, com valor padrão vazio. O valor padrão mantém compatíveis as chamadas antigas do construtor. O construtor continua verificando as condições básicas de um objeto Livro.

`domain/exemplares.ts` reúne Exemplar e Acervo. O Exemplar guarda seu estado e oferece métodos que controlam transições, como emprestar e devolver. O Acervo organiza livros e exemplares em memória e agora oferece atualização e remoção.

Ao atualizar um livro, seu ID é preservado, mantendo os exemplares associados. Ao remover um exemplar, o Acervo também libera o tombo registrado. Ao remover um livro, remove seus exemplares. As regras de bloqueio de exclusão da API são aplicadas por `ServicoAcervo` antes de chamar a remoção.

O contrato de armazenamento

`repositories/repositorio.ts` define `RepositorioAcervo`. A interface lista operações de leitura e escrita, incluindo listar, obter, salvar, atualizar e remover livros, adicionar e remover exemplares e verificar tombo existente.

`RepositorioMemoria` delega ao Acervo e permite simular latência ou falha. `RepositorioSQLite` executa SQL e reconstrói objetos de domínio com os dados retornados. Ambos atendem ao mesmo contrato, permitindo usar os mesmos serviços e testes.

No SQLite, uma linha usa a coluna `livro_id`, enquanto um objeto Exemplar usa `livroId`. A conversão do repositório resolve essa diferença. Métodos como `paraLivro` criam instâncias das classes a partir das linhas.

As consultas usam parâmetros. Em `WHERE id = ?`, o valor é passado separadamente ao método `get`. Isso evita montar a consulta concatenando a entrada recebida.

Os métodos retornam `Promise` porque essa é a interface adotada pelo projeto. `better-sqlite3` executa as operações de forma síncrona. A assinatura assíncrona mantém a compatibilidade com as demais implementações e com os chamadores.

Questões para responder

11. Quais responsabilidades pertencem ao Acervo e quais pertencem ao repositório SQLite?
12. Como o repositório transforma uma linha SQL em um Livro?
13. Por que `ServicoAcervo` recebe `RepositorioAcervo` no construtor?
14. Que operações de escrita foram adicionadas à interface?
15. Por que retornar Promise não torna automaticamente uma consulta SQLite não bloqueante?

5. Serviço e validação dos dados

`services/servico-acervo.ts` coordena as regras do CRUD. Ele verifica existência de recursos, ISBN repetido, pertencimento do exemplar ao livro e impedimentos de exclusão. Também cria IDs e monta os resumos devolvidos pela API.

`LivroResumo` contém os dados bibliográficos e as contagens `total` e `disponiveis`, calculadas pelos exemplares. `ExemplarResumo` expõe os dados da unidade e sua disponibilidade. Essas representações deixam explícito o formato entregue ao cliente.

Para cadastrar, o serviço valida o corpo, verifica ISBN livre, escolhe um ID, cria o Livro e pede ao repositório que salve. Para adicionar um exemplar, verifica o livro, valida o tombo e gera um código quando ele não foi informado.

Para excluir um livro, procura os exemplares e bloqueia a operação quando algum está emprestado ou reservado. Para consultar ou excluir um exemplar, confirma que ele pertence ao livro indicado na URL.

PUT e PATCH

`substituirLivro` atende ao PUT e exige os campos obrigatórios completos. Sinopse e capa omitidas passam a ser vazias. `atualizarLivro` atende ao PATCH e combina os valores atuais com os campos enviados, preservando os demais.

Por que validar em tempo de execução

`validacao-acervo.ts` recebe dados desconhecidos, verifica se são um objeto e confere cada campo. TypeScript verifica o código durante o desenvolvimento. Um cliente HTTP pode enviar qualquer conteúdo, então o programa precisa validá-lo ao receber.

A validação remove espaços nas pontas, rejeita campos desconhecidos, exige textos e números nos lugares esperados e verifica limites. `ano: "1899"` é texto e será rejeitado. `id` enviado pelo cliente é um campo não permitido.

O ISBN é normalizado, removendo caracteres fora de dígitos e X, e validado pelo formato de 10 ou 13 caracteres. A validação atual não calcula seu dígito verificador. Os IDs são escolhidos pelo maior número existente mais um. Requisições simultâneas ainda exigem revisão para evitar escolher o mesmo ID.

Questões para responder

16. Em qual etapa nasce o erro de ISBN repetido?
17. O que acontece com a sinopse omitida em um PUT? E em um PATCH?
18. Por que total e disponibilidade são calculados pelo servidor?
19. Por que o serviço confirma o pertencimento do exemplar ao livro?
20. Quais duas requisições simultâneas poderiam escolher o mesmo ID?

6. HTTP, controller e tratamento de erros

`routes/livros.routes.ts` associa método e caminho a uma função do controller. `GET /api/livros` lista, `POST /api/livros` cadastra e os caminhos com `:id` tratam um livro específico. Os exemplares usam a hierarquia `/api/livros/:id/exemplares/:exemplarId`.

`controllers/livros.controller.ts` lê parâmetros e corpo, chama o serviço e define a resposta HTTP. Uma criação devolve 201 e `Location`, indicando onde consultar o recurso. Uma exclusão bem-sucedida devolve 204 sem corpo.

Os handlers usam funções de seta, como `listar = async (...) => ...`. Assim preservam `this` ao serem passados para o Express. No Express 5, rejeições das funções assíncronas são encaminhadas ao tratamento de erros.

`domain/erros.ts` define `ErroValidacao`, `ErroNaoEncontrado` e `ErroConflito`. O serviço lança esses erros e `middlewares/tratar-erros.ts` converte cada tipo em status e JSON. A resposta segue o formato `{ "erro": "mensagem" }`.

- 200 - consulta ou atualização realizada.
- 201 - recurso criado, com Location.
- 204 - operação concluída sem corpo de resposta.
- 400 - dados inválidos ou JSON malformado.
- 404 - recurso ou rota inexistente.
- 405 - método não aceito naquele caminho.
- 409 - conflito com dados ou estado atual.
- 413 - corpo da requisição grande demais.
- 500 - falha inesperada, com mensagem genérica ao cliente e detalhe no log.

`metodo-nao-permitido.ts` devolve 405 com o cabeçalho `Allow`. OPTIONS recebe 204 com a lista dos métodos. HEAD funciona a partir do GET, devolvendo cabeçalhos sem o corpo. A configuração de CORS permite que o OPTIONS chegue a esse tratamento das rotas de livros.

As rotas de escrita ainda estão abertas. A etapa de autenticação deverá acrescentar os middlewares que exigem token e papel administrativo.

Questões para responder

21. Qual é a diferença entre erro 404 e 405? Dê uma requisição para cada caso.
22. Para que serve Location em uma criação?
23. Por que o cliente recebe uma mensagem genérica no erro 500?
24. Qual é a função dos quatro parâmetros do middleware de erros?
25. O que aconteceria com `this.servico` se o handler perdesse o vínculo com o controller?

7. Três requisições para rastrear no código

A. Consultar um livro

Envie `GET /api/livros/L01`. Em `app.ts`, a requisição entra no roteador montado em `/api/livros`. Em `livros.routes.ts`, a rota `/:id` chama `obter`. O controller transforma o parâmetro em texto e chama `servico.obterLivro`.

O serviço exige a existência do livro. O repositório executa uma consulta parametrizada na tabela `livros` e cria o objeto Livro. O serviço consulta os exemplares, calcula as contagens e monta o resumo. O controller envia o JSON com 200.

Se o repositório devolver ausência de livro, o serviço lança `ErroNaoEncontrado`. O middleware responde 404. A resposta de saúde é independente desse caminho e apenas devolve o status definido na rota.

B. Tentar cadastrar com corpo incompleto

Envie `POST /api/livros` com `{ "titulo": "Livro de estudo" }`. O JSON é válido e chega ao controller. O serviço chama `lerLivroCompleto`, que exige os campos obrigatórios. A ausência do primeiro campo obrigatório não preenchido lança `ErroValidacao`.

O middleware responde 400. A operação não chega à inserção no repositório. Compare com um corpo de sintaxe inválida, como `{ "titulo": }`, que falha ao interpretar o JSON.

C. Tentar excluir um livro com exemplar emprestado

Envie `DELETE /api/livros/L01` no acervo inicial. O controller chama o serviço, que confirma a existência do livro e lista seus exemplares. Como o conjunto inicial contém exemplar emprestado, lança `ErroConflito`.

O middleware responde 409 e a remoção do livro não é executada. O CASCADE só teria efeito se a exclusão chegasse ao banco.

Registro para preencher

Para cada fluxo, responda às mesmas perguntas.

- Quais arquivos foram percorridos, em ordem?
- Qual dado entrou em cada função?
- Qual função tomou a decisão principal?
- Houve consulta ou gravação no banco?
- Qual foi o status final e qual camada o definiu?

Tente explicar cada fluxo em dois minutos. Depois confira os nomes reais das funções no projeto.

8. Testes e estado verificado do projeto

Em 05/11, os typechecks do server e do web passaram. No back-end, passaram 277 testes em 9 arquivos. A verificação HTTP de inicialização retornou 200 em saúde e catálogo, usando SQLite em memória, com 9 livros e 19 exemplares.

`tests/api.test.ts` usa Supertest para enviar requisições à aplicação. A suíte principal roda com repositório em memória e com SQLite temporário. Isso compara o comportamento HTTP usando as duas implementações. Os testes verificam corpo, status e cabeçalhos, incluindo Location e Allow.

`tests/banco.test.ts` verifica criação das tabelas, seed, unicidade, chaves estrangeiras e status válidos. `acervoEscrita.test.ts` cobre operações adicionadas ao Acervo. Os testes de repositório e dados iniciais também foram atualizados. Os testes antigos do domínio e empréstimos continuam fazendo parte da suíte.

Comandos dentro de server/

```
npm run typecheck
npm test
```

A primeira execução HTTP no ambiente da análise foi bloqueada por restrição de conexões locais. Ao permitir as conexões e executar o comando abaixo, todos os testes passaram. Use essa opção também para comparar o aviso de encerramento observado na execução padrão.

```
npm test -- --pool=threads --maxWorkers=1
```

Uma suíte verde confirma os casos cobertos. A persistência em arquivo após reiniciar, a concorrência na geração de IDs e o fluxo pelo navegador precisam de verificações próprias. Os testes de SQLite atuais usam principalmente bancos temporários.

O front ainda tem cards estáticos e o Vite está sem proxy `/api`. O build terminou com aviso de aspas no atributo de sinopse em `web/acervo.html`, linha 273. As tabelas das próximas funcionalidades já existem, mas cadastro, login e endpoints de empréstimos e reservas seguem pendentes.

Questões para responder

26. Qual diferença existe entre testar uma regra do Acervo e testar uma rota com Supertest?
27. Por que a suíte cria uma aplicação nova para cada teste?
28. O que executar a suíte com dois repositórios ajuda a detectar?
29. Qual teste você faria para verificar persistência em arquivo?
30. O que falta entre ter a API funcionando e mostrar seus livros na página?

9. Exercícios práticos guiados

Execute a API em um terminal com `cd server` e `npm run dev`. Em outro terminal, envie as requisições. Use dados de desenvolvimento. Registre o resultado antes de consultar a explicação novamente.

Exercício 1 - consulta e erro de existência

```
Invoke-RestMethod http://localhost:3000/api/saude
$livros = Invoke-RestMethod http://localhost:3000/api/livros
$livros.Count
Invoke-RestMethod http://localhost:3000/api/livros/L01
curl.exe -i http://localhost:3000/api/livros/NAO-EXISTE
```

Compare a consulta correta com a inexistente. Localize no serviço o ponto que gera o 404. Explique como total e disponíveis são calculados.

Exercício 2 - criar, alterar e excluir seu livro de teste

```
$corpo = @{
  titulo = 'Livro de estudo'
  autor = 'Autora de teste'
  isbn = '9781234567897'
  ano = 2026
  genero = 'Estudo'
} | ConvertTo-Json
$novo = Invoke-RestMethod http://localhost:3000/api/livros `
  -Method Post -ContentType 'application/json' -Body $corpo
$url = "http://localhost:3000/api/livros/$($novo.id)"
Invoke-RestMethod $url -Method Patch `
  -ContentType 'application/json' -Body '{"sinopse":"Experimento"}'
Invoke-RestMethod $url
Invoke-RestMethod $url -Method Delete
```

O ISBN precisa estar livre. Se repetir a criação antes de excluir o livro de teste, espere um conflito. Anote o ID realmente retornado, sem presumir que será sempre L10. Para ver os cabeçalhos e o 201, abra também o teste correspondente de Supertest.

Exercício 3 - validar sem cadastrar

```
curl.exe -i -X POST http://localhost:3000/api/livros `
  -H 'Content-Type: application/json' --data-binary '{}'
curl.exe -i -X DELETE http://localhost:3000/api/livros
```

Preveja os status antes de executar. No primeiro caso, encontre o campo obrigatório apontado. No segundo, leia Allow e explique por que o caminho é reconhecido. Rode os testes ao terminar qualquer alteração experimental.

10. Revisão final e critérios para conferir as respostas

Responda às questões anteriores com suas palavras. Depois use estes critérios para conferir se sua explicação inclui os elementos essenciais. Volte ao código quando houver dúvida.

- Inicialização - separar a montagem da aplicação da abertura da porta permite testar a aplicação com diferentes repositórios. A rota de saúde atual retorna um objeto fixo.
- Banco - livro e exemplar representam entidades distintas. Chaves estrangeiras protegem os relacionamentos, e a transação agrupa a população inicial. O seed só roda quando não há livros.
- Repositórios - a interface define o contrato, as implementações realizam o armazenamento e o serviço recebe esse contrato por injeção de dependência.
- Regras - ISBN repetido e exclusão em uso são verificados no serviço. A validação do corpo rejeita entradas inválidas antes de gravar. PUT substitui e PATCH combina os campos enviados com os atuais.
- HTTP - o controller escolhe a resposta de sucesso. Os erros conhecidos são convertidos no middleware. Location aponta para o recurso criado, e Allow informa métodos aceitos.
- Testes - testes HTTP percorrem mais camadas que testes isolados do domínio. Criar dados e aplicação novos reduz interferência entre os casos. Testar persistência exige fechar e reabrir o mesmo banco em arquivo.

Desafios para responder sem consultar

31. Desenhe o caminho completo de uma consulta ao livro L01 e indique onde entra SQL.
32. Explique por que um cadastro pode falhar com 400 ou 409.
33. Dê um exemplo concreto de encapsulamento, um de polimorfismo e um de injeção de dependência neste projeto.
34. Explique o que ocorre se você reiniciar a API e depois o que ocorre se executar db:reset.
35. Escolha um teste de API, leia suas expectativas e explique qual regressão ele detecta.
36. Liste três pendências que os 277 testes aprovados ainda não resolvem.

Checklist ao terminar

[ ] Expliquei a inicialização sem consultar o guia.

[ ] Diferenciei livro, exemplar e empréstimo.

[ ] Rastreiei os três fluxos e a origem de seus status.

[ ] Executei os exercícios e registrei os resultados.

[ ] Entendi o efeito do reset antes de considerar seu uso.

[ ] Identifiquei o que já funciona e o que falta conectar no front.

[ ] Anotei dúvidas com arquivo e função específicos.

Arquivos de referência

Todos os caminhos citados são relativos à raiz `D:/PROJETOS/Biblioteca-Virtual`. Abra os arquivos no editor seguindo os caminhos indicados em cada capítulo. O conteúdo deste guia se baseia na leitura do código e nas verificações de 05/11/2026. Reconfira as observações quando a implementação mudar.